

EECE7205 Assignment01

Ayham Makhamra

September 2026

Each test case presented in this report was also added to the `main()` function in the C++ implementation and tested to verify the correctness of the solution.

1 Problem 1: Merge Intervals

1.1 1. Understand the Problem

Input:

A list of intervals, where each interval is represented as:

$$[start, end]$$

Output:

A list of merged, non-overlapping intervals sorted by their starting values.

Requirements:

- Sort the intervals by their starting values in ascending order.
- If the next interval starts before or when the current interval ends, merge them.
- Use the smaller start and the larger end for the merged interval.
- Keep non-overlapping intervals separate.
- Do not lose or duplicate any interval coverage.

1.2 2. Design the Solution

- Sort the intervals by their starting values in ascending order.
- Compare each interval only with the last merged interval.
- If $next.start \leq current.end$, the intervals overlap.

1.3 3. Pseudocode

Algorithm 1 Merge Overlapping Intervals

```
1: function MERGEINTERVALS(intervals)
2:   if intervals is empty then
3:     return empty list
4:   end if
5:   Sort intervals by starting value
6:   merged  $\leftarrow$  empty list
7:   current  $\leftarrow$  first interval
8:   for each next interval starting from the second interval do
9:     if next.start  $\leq$  current.end then
10:      current.end  $\leftarrow$  max(current.end, next.end)
11:    else
12:      Add current to merged
13:      current  $\leftarrow$  next
14:    end if
15:  end for
16:  Add current to merged
17:  return merged
18: end function
```

1.4 4. Correctness

The algorithm correctly merges all overlapping intervals because the intervals are first sorted by their starting values. Therefore, when processing an interval, all intervals that could overlap with the current interval have already been considered or are being considered next.

For each next interval, the algorithm checks whether

$$\textit{next.start} \leq \textit{current.end}.$$

If this condition is true, the intervals overlap, so the algorithm extends the current interval to include the larger ending value. This preserves all interval coverage.

If

$$\textit{next.start} > \textit{current.end},$$

the intervals do not overlap. Therefore, the current interval can be added to the result without merging it with the next interval.

Because every interval is processed exactly once after sorting, no interval is lost or duplicated. The final current interval is also added after the loop, ensuring that the last merged interval is included in the result.

The algorithm also handles an empty input by immediately returning an empty list.

1.5 5. Complexity Analysis

Let n be the number of intervals.

Time Complexity:

The intervals are first sorted by their starting values. Comparison-based sorting takes

$$O(n \log n)$$

time.

After sorting, the algorithm makes one pass through the intervals, which takes

$$O(n)$$

time.

Therefore, the overall time complexity is

$$O(n \log n).$$

The sorting step dominates the linear merging step.

Best Case:

The sorting operation still requires

$$O(n \log n)$$

time for the comparison-based sorting used by the implementation. The merging pass takes $O(n)$ time.

Therefore, the overall best-case complexity is

$$O(n \log n).$$

Average Case:

The sorting step takes

$$O(n \log n)$$

time on average, followed by an $O(n)$ merging pass. Thus,

$$O(n \log n).$$

Worst Case:

The sorting step takes

$$O(n \log n)$$

time, followed by an $O(n)$ merging pass. Therefore,

$$O(n \log n).$$

Space Complexity:

The algorithm requires space for the resulting merged intervals. In the worst case, when none of the intervals overlap, the result contains n intervals.

Therefore, the auxiliary/output space is

$$O(n).$$

The exact auxiliary space used by the sorting operation depends on the sorting implementation.

1.6 6. C++ Implementation

The C++ implementation represents each interval as a `vector<int>` containing its start and end values. The intervals are first sorted according to their starting values. The merging procedure then follows the pseudocode directly by maintaining the current interval and comparing it with each subsequent interval.

The implementation is included in the assignment's C++ program, where the solution is organized as a separate function for Problem 1.

1.7 7. Testing

The solution was tested using multiple inputs to verify normal cases, non-overlapping intervals, overlapping intervals, touching intervals, and empty input.

1.7.1 Test Case 1: Overlapping Intervals

Input:

$[[1, 3], [2, 6], [8, 10], [15, 18]]$

Expected Output:

$[[1, 6], [8, 10], [15, 18]]$

The first two intervals overlap and are merged into $[1, 6]$. The remaining intervals do not overlap and remain separate.

1.7.2 Test Case 2: Touching Intervals

Input:

$[[1, 4], [4, 5]]$

Expected Output:

$[[1, 5]]$

The intervals are merged because the starting value of the second interval is equal to the ending value of the first interval:

$$4 \leq 4.$$

1.7.3 Test Case 3: Non-Overlapping Intervals

Input:

$[[1, 2], [4, 5], [7, 9]]$

Expected Output:

$[[1, 2], [4, 5], [7, 9]]$

None of the intervals overlap, so each interval remains separate.

1.7.4 Edge Case: Empty Input

Input:

`[]`

Expected Output:

`[]`

The algorithm detects that the input is empty and returns an empty list without attempting to access the first interval.

2 Problem 2: Sort 0, 1, and 2

2.1 1. Understand the Problem

Input:

An array containing only the numbers 0, 1, and 2.

Output:

The same array sorted in place so that all 0s appear first, followed by all 1s, and then all 2s.

- The input contains only the values 0, 1, and 2.
- The array must be sorted in place.
- The solution must not use `std::sort()`.
- Two approaches must be implemented: a counting-based solution and the one-pass Dutch National Flag algorithm.
- All 0s should appear before all 1s, and all 1s should appear before all 2s.
- The algorithm should correctly handle an empty array, an array containing only one of the three values, and arrays where one or more of the values do not occur.

2.2 2. Design the Solution

Two approaches are used to solve this problem: a counting-based solution and the one-pass Dutch National Flag algorithm.

2.2.1 Counting-Based Solution

The counting-based solution uses an auxiliary array of size 3 to store the number of occurrences of each value. The indices of this array correspond to the three possible values in the input:

`count[0], count[1], count[2].`

First, the algorithm loops through the original array and increments the corresponding count for each element. After all elements have been counted, the algorithm goes through the count array and overwrites the original array using the recorded frequencies.

Because there are only three possible values, the count array always has a fixed size of 3. The original array is therefore modified in place after the counting step.

2.2.2 Dutch National Flag Solution

The Dutch National Flag algorithm sorts the array in a single pass by dividing the array into three regions: elements containing 0, elements that have not yet been processed, and elements containing 2.

The algorithm maintains three indices: **low**, **mid**, and **high**. The **mid** index is used to examine the current element.

When the current element is 0, it belongs in the left region, so it is swapped with the element at **low**. Both **low** and **mid** are then advanced.

When the current element is 1, it is already in the correct middle region, so only **mid** is advanced.

When the current element is 2, it belongs in the right region, so it is swapped with the element at **high** and **high** is decreased. The **mid** index is not advanced in this case because the element swapped into the current position has not yet been examined.

2.3 3. Pseudocode

2.3.1 Counting-Based Solution

Algorithm 2 Sort 0, 1, and 2 Using Counting

```

1: function COUNTINGSORT012(array)
2:   count  $\leftarrow$  {0,0,0}
3:   for each element in array do
4:     count[element]  $\leftarrow$  count[element] + 1
5:   end for
6:   index  $\leftarrow$  0
7:   for value  $\leftarrow$  0 to 2 do
8:     for i  $\leftarrow$  1 to count[value] do
9:       array[index]  $\leftarrow$  value
10:      index  $\leftarrow$  index + 1
11:    end for
12:  end for
13:  return array
14: end function

```

Algorithm 3 Dutch National Flag Algorithm

```
1: function DUTCHNATIONALFLAG(array)
2:   low  $\leftarrow$  0
3:   mid  $\leftarrow$  0
4:   high  $\leftarrow$  length of array - 1
5:   while mid  $\leq$  high do
6:     if array[mid] = 0 then
7:       Swap array[low] and array[mid]
8:       low  $\leftarrow$  low + 1
9:       mid  $\leftarrow$  mid + 1
10:    else if array[mid] = 1 then
11:      mid  $\leftarrow$  mid + 1
12:    else
13:      Swap array[mid] and array[high]
14:      high  $\leftarrow$  high - 1
15:    end if
16:  end while
17:  return array
18: end function
```

2.3.2 Dutch National Flag Solution

2.4 4. Correctness

2.4.1 Counting-Based Solution

The counting-based solution correctly sorts the array because it first records the exact number of occurrences of each possible value. Since the input can only contain 0, 1, and 2, every element is counted exactly once.

After counting, the algorithm overwrites the original array by writing all 0s first, followed by all 1s, and then all 2s. The number of times each value is written is exactly equal to its recorded frequency.

Therefore, the resulting array contains the same elements as the original array, with all values arranged in the required order. No elements are lost or duplicated.

If the input is empty, the counting loop performs no iterations and the reconstruction step also writes no elements, leaving the array empty.

2.4.2 Dutch National Flag Solution

The Dutch National Flag algorithm maintains three regions during execution:

- Values before **low** are all 0s.
- Values between **low** and **mid** are all 1s.
- Values after **high** are all 2s.

The region between `mid` and `high` contains the elements that have not yet been fully processed.

If the current element is 0, it is swapped into the 0 region and both `low` and `mid` are advanced.

If the current element is 1, it is already in the correct middle region, so `mid` is advanced.

If the current element is 2, it is swapped into the 2 region and `high` is decreased. The `mid` index is not advanced because the element swapped into the current position has not yet been examined.

The algorithm terminates when `mid` passes `high`. At that point, all elements have been placed into their correct regions, so the array is sorted as 0s, followed by 1s, followed by 2s.

2.5 5. Complexity Analysis

Let n be the number of elements in the array.

2.5.1 Counting-Based Solution

Time Complexity:

The first loop processes every element in the input array, which takes

$$O(n)$$

time.

The reconstruction uses an outer loop over the three possible values 0, 1, and 2. This loop has a fixed size and therefore takes

$$O(1)$$

time.

The inner reconstruction loops collectively write exactly n elements back into the array. Therefore, they take

$$O(n)$$

time in total.

Thus, the overall time complexity is

$$O(n).$$

Although the reconstruction contains nested loops, the outer loop always has a fixed size of 3. Therefore, the nested structure does not result in $O(n^2)$ complexity.

Best Case:

$$O(n)$$

Average Case:

$$O(n)$$

Worst Case:

$$O(n)$$

The counting approach has the same asymptotic time complexity regardless of the distribution of 0s, 1s, and 2s.

Space Complexity:

The algorithm uses a count array of fixed size 3.

Therefore, the auxiliary space complexity is

$$O(1).$$

2.5.2 Dutch National Flag Solution

Time Complexity:

The Dutch National Flag algorithm processes the array using the three pointers `low`, `mid`, and `high`. Each element is processed a constant number of times, and the pointers move toward the end of the array.

Therefore, the overall time complexity is

$$O(n).$$

Best Case:

$$O(n)$$

Average Case:

$$O(n)$$

Worst Case:

$$O(n)$$

Regardless of the arrangement of the input values, the algorithm processes the elements using a linear number of operations.

Space Complexity:

The algorithm uses only the three variables `low`, `mid`, and `high`, in addition to a constant amount of temporary space for swapping.

Therefore, the auxiliary space complexity is

$$O(1).$$

2.6 6. C++ Implementation

Both solutions are implemented as separate functions in the C++ program.

The counting-based implementation first records the frequency of each value using a fixed-size array of three elements. It then overwrites the original array using those frequencies.

The Dutch National Flag implementation uses the three pointers `low`, `mid`, and `high` to sort the array in one pass without using `std::sort()`.

Both implementations modify the input array in place and use $O(1)$ auxiliary space.

2.7 7. Testing

Both solutions were tested using inputs containing different combinations of 0s, 1s, and 2s. The test cases include arrays with missing categories, an array containing only one category, and an empty array.

2.7.1 Test Case 1: Mixed Values

Input:

[2, 0, 2, 1, 1, 0]

Expected Output:

[0, 0, 1, 1, 2, 2]

This test case contains all three possible values in an unsorted order.

2.7.2 Test Case 2: No 2s

Input:

[0, 1, 1, 0, 1, 0]

Expected Output:

[0, 0, 0, 1, 1, 1]

This test case verifies that both algorithms work correctly when one of the three categories does not occur.

2.7.3 Test Case 3: Only One Category

Input:

[2, 2, 2, 2]

Expected Output:

[2, 2, 2, 2]

This test case verifies that the algorithms correctly handle an array in which every element already belongs to the same category.

2.7.4 Edge Case: Empty Input

Input:

[]

Expected Output:

[]

The input contains no elements, so both algorithms should return without performing any sorting operations.

3 Problem 3: Largest Number

3.1 1. Understand the Problem

Input:

A vector of nonnegative integers.

Output:

A string representing the largest possible number that can be formed by concatenating all the given integers.

Requirements:

- Arrange all numbers in an order that produces the largest possible concatenated number.
- Numbers must be compared based on the result of concatenating them in different orders.
- The final result must be returned as a string because the resulting number may be too large to store in a standard integer type.
- If all input values are zero, return a single “0” instead of a string containing multiple zeros.

Key Comparison: For two numbers a and b , compare the two possible concatenations:

$$ab \text{ and } ba.$$

If $ab > ba$, then a should appear before b . Otherwise, b should appear before a .

Edge Cases: The solution should handle an empty input and an input containing only zeros.

3.2 2. Design the Solution

The solution uses a repeated-selection approach. Since the goal is to create the largest possible concatenated number, the normal numerical order of the numbers is not sufficient.

For example, consider the numbers 9 and 34. We compare the two possible orders:

$$934 \text{ and } 349.$$

Since 934 is larger, 9 should be placed before 34.

The algorithm first converts all integers to strings so that they can be concatenated and compared easily. It then repeatedly searches through the remaining numbers to find the number that should be placed next.

For each candidate, the algorithm compares:

$$candidate + currentBest$$

with

$$currentBest + candidate.$$

If placing the candidate first produces a larger concatenation, the candidate becomes the new best choice.

Once the best number is found, it is appended to the result and removed from the list of remaining numbers. This process continues until all numbers have been selected.

Finally, if the resulting string begins with “0”, all input values must have been zero. In this case, the algorithm returns a single “0”.

3.3 3. Pseudocode

Algorithm 4 Construct the Largest Number

```
1: function LARGESTNUMBER(numbers)
2:   numberStrings  $\leftarrow$  empty list
3:   for each number in numbers do
4:     Convert number to a string
5:     Add the string to numberStrings
6:   end for
7:   if numberStrings is empty then
8:     return empty string
9:   end if
10:  result  $\leftarrow$  empty string
11:  while numberStrings is not empty do
12:    bestIndex  $\leftarrow$  0
13:    for i  $\leftarrow$  1 to length of numberStrings - 1 do
14:      firstOrder  $\leftarrow$  numberStrings[i] + numberStrings[bestIndex]
15:      secondOrder  $\leftarrow$  numberStrings[bestIndex] + numberStrings[i]
16:      if firstOrder > secondOrder then
17:        bestIndex  $\leftarrow$  i
18:      end if
19:    end for
20:    Append numberStrings[bestIndex] to result
21:    Remove numberStrings[bestIndex]
22:  end while
23:  if the first character of result is “0” then
24:    return “0”
25:  end if
26:  return result
27: end function
```

3.4 4. Correctness

The algorithm determines the correct order by comparing every candidate with the number currently considered the best choice.

For two numbers a and b , the algorithm compares ab and ba . If $ab > ba$, placing a before b produces a larger result than placing b before a . Therefore, the comparison determines which of the two numbers should appear first.

At each iteration, the algorithm examines all remaining numbers and selects the number that should be placed next according to this comparison rule. The selected number is then permanently added to the result and removed from the remaining list.

This process continues until every number has been selected. Therefore, every input number appears exactly once in the final result.

The algorithm also handles the special case where all numbers are zero. In that case, the concatenated result consists only of zeros, so the algorithm returns a single “0”.

3.5 5. Complexity Analysis

Let n be the number of input integers, and let m represent the average number of digits in the integers. Or we can simply use the total number of digits in the array, but for simplicity let's proceed with the average number.

Time Complexity:

Converting the n integers to strings requires

$$O(nm)$$

time.

The main loop selects one number at a time. During the first iteration, it compares approximately n numbers. During the next iteration, approximately $n - 1$ numbers remain, then $n - 2$, and so on.

Therefore, the number of comparisons is

$$n + (n - 1) + (n - 2) + \cdots + 1 = O(n^2).$$

Each comparison involves concatenating and comparing strings. If the average number of digits is m , this requires approximately $O(m)$ time.

Therefore, the selection process takes

$$O(n^2m)$$

time.

In addition, removing an element from a vector can require shifting the remaining elements, which can take $O(n)$ time in the worst case. Across all removals, this contributes another $O(n^2)$ operations.

Thus, the overall time complexity is

$$O(n^2m)$$

when accounting for the length of the numbers.

If the number of digits is treated as bounded by a constant, the complexity can be expressed more simply as

$$O(n^2).$$

Best Case:

The algorithm still examines the remaining numbers at every selection step, so it performs a quadratic number of comparisons:

$$O(n^2m)$$

or $O(n^2)$ when string lengths are treated as constant.

Average Case:

The algorithm performs a quadratic number of comparisons:

$$O(n^2m).$$

Worst Case:

The algorithm performs a quadratic number of comparisons and may also shift elements when removing them from the vector:

$$O(n^2m).$$

Space Complexity:

The algorithm creates a vector of strings containing all n input numbers. The space required to store these strings is

$$O(nm).$$

The result string also contains all input digits and therefore requires $O(nm)$ space.

Thus, the overall space complexity is

$$O(nm).$$

When the number of digits is treated as bounded by a constant, this can be expressed as

$$O(n).$$

3.6 6. C++ Implementation

The implementation first converts each integer into a string and stores the strings in a vector.

At each iteration, the algorithm searches for the number that should be placed next. It compares the two possible concatenation orders and keeps track of the index of the best candidate.

Once the best candidate is found, it is added to the result and removed from the vector of remaining numbers.

The implementation also checks for an empty input and handles the special case where all input values are zero.

The C++ implementation follows the pseudocode directly and uses clear variable names such as `numberStrings`, `bestIndex`, `firstOrder`, `secondOrder`, and `result`.

3.7 7. Testing

The solution was tested using inputs containing numbers with different digit lengths, numbers that require the concatenation comparison to determine their order, and special cases such as an empty input and inputs containing only zeros.

3.7.1 Test Case 1: Different Digit Lengths

Input:

[10, 2]

Expected Output:

“210”

The algorithm compares 210 with 102 and therefore places 2 before 10.

3.7.2 Test Case 2: Multiple Numbers

Input:

[3, 30, 34, 5, 9]

Expected Output:

“9534330”

The result is formed by ordering the numbers according to the concatenation comparison rather than their normal numerical order.

3.7.3 Test Case 3: Repeated Values

Input:

[12, 121]

Expected Output:

“12121”

The algorithm compares 12121 with 12112 and places 12 before 121.

3.7.4 Edge Case: All Zeros

Input:

`[0, 0, 0]`

Expected Output:

`"0"`

Although concatenating the values would produce `"000"`, the algorithm returns a single `"0"` as required.

3.7.5 Edge Case: Empty Input

Input:

`[]`

Expected Output:

`""`

The algorithm checks whether the converted string vector is empty and returns an empty string without attempting to access the first character.

4 Problem 4: Group Anagrams

4.1 1. Understand the Problem

Input:

A vector of strings.

Output:

A collection of groups where each group contains strings that are anagrams of one another.

The order of the groups does not matter, but the strings within each group must retain their original input order.

Requirements:

- Strings containing the same characters with the same frequencies should be placed in the same group.
- The order of the groups does not matter.
- The original order of strings within each anagram group must be preserved.
- Each input string must appear exactly once in the result.

Key Idea: Anagrams become identical when their characters are sorted. For example,

`"eat" → "aet"`

`"tea" → "aet"`

"ate" $\rightarrow$ "aet".

Therefore, the sorted version of a string can be used as a key to identify its anagram group.

4.2 2. Design the Solution

The solution uses an unordered hash map. Each key is the sorted version of a string, and the corresponding value is a vector containing the original strings that have that sorted form.

The algorithm processes the input strings one at a time. For each string, it creates a copy and sorts the characters in that copy. The sorted string is then used as the key in the hash map.

The original string is added to the vector associated with that key. Because the input strings are processed from beginning to end, strings within each group are automatically kept in their original input order.

After all strings have been processed, the vectors stored in the hash map are collected into the final result.

4.3 3. Pseudocode

Algorithm 5 Group Anagrams

```
1: function GROUPANAGRAMS(strings)
2:   anagramGroups  $\leftarrow$  empty hash map
3:   for each string in strings do
4:     sortedString  $\leftarrow$  copy of string
5:     Sort characters of sortedString
6:     Add string to anagramGroups[sortedString]
7:   end for
8:   result  $\leftarrow$  empty list
9:   for each group in anagramGroups do
10:    Add group to result
11:   end for
12:   return result
13: end function
```

4.4 4. Correctness

The algorithm correctly groups anagrams because two strings are anagrams if and only if they contain the same characters with the same frequencies. Sorting the characters of each string produces the same sorted string for anagrams.

For example, the strings "eat", "tea", and "ate" all produce the key "aet". Therefore, they are added to the same vector in the hash map.

Strings that are not anagrams produce different sorted keys and are therefore placed into different groups.

Each original string is processed exactly once and added to exactly one group. Since the strings are processed in their original input order, the order of strings within each group is also preserved.

Therefore, the algorithm produces groups containing exactly the required anagrams without losing or duplicating any input string.

4.5 5. Complexity Analysis

Let m be the number of strings and let n be the length of the longest string.

Time Complexity:

For each of the m strings, the algorithm creates a copy and sorts its characters.

Sorting a string of length n takes

$$O(n \log n)$$

time in the worst case.

Therefore, sorting all m strings takes

$$O(mn \log n).$$

The hash map insertion and lookup operations take expected $O(1)$ time, so they do not change the overall asymptotic complexity.

Thus, the overall expected time complexity is

$$O(mn \log n).$$

Best Case:

The strings still need to be processed and their characters sorted, so the overall complexity remains

$$O(mn \log n).$$

Average Case:

The sorting of each string takes $O(n \log n)$ time, giving

$$O(mn \log n).$$

Worst Case:

The sorting operations take $O(n \log n)$ per string, giving

$$O(mn \log n).$$

Space Complexity:

The hash map stores all input strings in their corresponding groups. The sorted copies used as keys also require space.

Therefore, the space required to store the grouped strings and keys is

$$O(mn).$$

The additional space used for sorting an individual string is at most $O(n)$, which is dominated by the storage of all strings when there are multiple input strings.

Thus, the overall space complexity is

$$O(mn).$$

4.6 6. C++ Implementation

The C++ implementation uses an `unordered_map` where the key is the sorted version of each string and the value is a vector containing the original strings belonging to that group.

For each input string, the implementation creates a copy, sorts the characters of the copy, and uses the sorted copy as the hash-map key. The original string is then added to the corresponding group.

Finally, the groups stored in the hash map are copied into a result vector and returned.

The implementation preserves the original order of strings within each group because strings are inserted into each group in the same order in which they appear in the input.

4.7 7. Testing

The solution was tested using strings containing multiple groups of anagrams, strings with no anagrams, repeated anagrams, and an empty input.

4.7.1 Test Case 1: Multiple Anagram Groups

Input:

```
["eat", "tea", "tan", "ate", "nat", "bat"]
```

Expected Output:

```
[["eat", "tea", "ate"], ["tan", "nat"], ["bat"]]
```

The strings "eat", "tea", and "ate" all have the sorted key "aet", while "tan" and "nat" both have the sorted key "ant".

4.7.2 Test Case 2: No Anagrams

Input:

```
["cat", "dog", "bird"]
```

Expected Output:

```
[["cat"], ["dog"], ["bird"]]
```

Each string has a different sorted key, so every string forms its own group.

4.7.3 Test Case 3: Repeated Anagrams

Input:

```
["listen", "silent", "enlist", "listen"]
```

Expected Output:

```
[["listen", "silent", "enlist", "listen"]]
```

All four strings have the same characters with the same frequencies, so they belong to the same group. Their original input order is preserved.

4.7.4 Edge Case: Empty Input

Input:

```
[]
```

Expected Output:

```
[]
```

The algorithm processes no strings and returns an empty result.

5 Problem 5: Meeting Overlap

5.1 1. Understand the Problem

Input:

A list of meeting intervals, where each meeting is represented as:

```
[start, end]
```

Output:

A Boolean value indicating whether one person can attend every meeting.

- Return **true** if the person can attend all meetings.
- Return **false** if any two meetings overlap.

Requirements:

- Sort the meetings by their starting values in ascending order.
- Compare each meeting with the previous meeting.
- If the next meeting starts before the current meeting ends, the meetings overlap.
- Meetings that begin exactly when another meeting ends are allowed.

Overlap condition:

Two meetings overlap if

$$next.start < current.end.$$

If

$$next.start = current.end,$$

the meetings do not overlap because the first meeting has ended exactly when the next meeting begins.

5.2 2. Design the Solution

The solution first sorts all meeting intervals by their starting values in ascending order. After sorting, meetings that could overlap will appear next to each other.

The algorithm then compares each meeting with the previous meeting. If the starting time of the next meeting is less than the ending time of the current meeting, the two meetings overlap, so the person cannot attend both meetings. In this case, the algorithm immediately returns **false**.

If no overlap is found after checking all consecutive meetings, then the person can attend every meeting, so the algorithm returns **true**.

Because the meetings are sorted first, it is sufficient to compare each meeting with the previous meeting rather than comparing every pair of meetings.

5.3 3. Pseudocode

Algorithm 6 Determine Whether All Meetings Can Be Attended

```
1: function CANATTENDMEETINGS(meetings)
2:   if meetings is empty then
3:     return true
4:   end if
5:   Sort meetings by starting value
6:   for  $i \leftarrow 1$  to length of meetings - 1 do
7:      $current \leftarrow meetings[i - 1]$ 
8:      $next \leftarrow meetings[i]$ 
9:     if  $next.start < current.end$  then
10:      return false
11:    end if
12:  end for
13:  return true
14: end function
```

5.4 4. Correctness

The algorithm first sorts all meetings by their starting times. After sorting, if two meetings overlap, the earlier-starting meeting will appear before the later-starting meeting.

The algorithm then checks every pair of consecutive meetings. For each pair, it compares the starting time of the next meeting with the ending time of the current meeting.

If

$$next.start < current.end,$$

then the next meeting begins before the current meeting has ended. Therefore, the two meetings overlap, and one person cannot attend both meetings. The algorithm correctly returns **false**.

If

$$next.start \geq current.end,$$

then the two meetings do not overlap. In particular, when

$$next.start = current.end,$$

the meetings occur back-to-back and are allowed by the problem.

If the algorithm finishes checking every consecutive pair without finding an overlap, then no two meetings conflict. Therefore, the person can attend every meeting and the algorithm correctly returns **true**.

Thus, the algorithm correctly determines whether all meetings can be attended.

5.5 5. Complexity Analysis

Let n be the number of meetings.

Time Complexity:

The first step sorts the meetings by their starting values. Sorting n intervals takes

$$O(n \log n)$$

time.

After sorting, the algorithm makes one pass through the meetings to check for overlapping intervals. This takes

$$O(n)$$

time.

Therefore, the total time complexity is

$$O(n \log n) + O(n).$$

Since the sorting step dominates the linear scan, the overall time complexity is

$$O(n \log n).$$

Best Case:

If an overlap is found early in the scan, the algorithm can stop before checking every meeting. However, the meetings still need to be sorted first, so the overall complexity remains

$$O(n \log n).$$

Average Case:

The meetings are sorted and then scanned once, giving

$$O(n \log n).$$

Worst Case:

If there are no overlapping meetings, the algorithm must scan the entire sorted list after sorting it. Therefore,

$$O(n \log n).$$

Space Complexity:

The algorithm only uses a constant amount of additional space for variables such as the current and next intervals.

Therefore, the auxiliary space complexity is

$$O(1)$$

assuming the sorting operation is considered separately from the algorithm's auxiliary variables.

5.6 6. C++ Implementation

The C++ implementation first sorts the meeting intervals according to their starting times. It then checks each meeting against the previous meeting.

If the next meeting starts before the current meeting ends, the function immediately returns `false`. Otherwise, the algorithm continues checking the remaining meetings.

If all meetings are checked without finding an overlap, the function returns `true`.

The implementation uses the strict inequality

$$meetings[i][0] < meetings[i - 1][1]$$

because meetings that start exactly when another meeting ends are allowed.

5.7 7. Testing

The solution was tested using overlapping meetings, non-overlapping meetings, back-to-back meetings, and edge cases such as an empty input.

5.7.1 Test Case 1: Overlapping Meetings

Input:

`[[0, 30], [5, 10], [15, 20]]`

Expected Output:

`false`

After sorting, the meetings are

`[[0, 30], [5, 10], [15, 20]].`

The second meeting starts at 5, which is before the first meeting ends at 30. Therefore, the meetings overlap.

5.7.2 Test Case 2: Non-Overlapping Meetings

Input:

`[[5, 10], [15, 20], [25, 30]]`

Expected Output:

`true`

Each meeting begins after the previous meeting has ended, so the person can attend all meetings.

5.7.3 Test Case 3: Back-to-Back Meetings

Input:

`[[1, 3], [3, 5], [5, 8]]`

Expected Output:

`true`

Each meeting starts exactly when the previous meeting ends. Since back-to-back meetings are allowed, there is no overlap.

5.7.4 Test Case 4: Unsorted Input

Input:

`[[15, 20], [0, 10], [5, 15]]`

Expected Output:

`false`

After sorting, the meetings become

$[[0, 10], [5, 15], [15, 20]]$.

The first two meetings overlap because the second meeting starts at 5, before the first meeting ends at 10.

5.7.5 Edge Case: Empty Input

Input:

$[]$

Expected Output:

`true`

There are no meetings that conflict, so the person can attend all meetings.

6 Problem 6: K Closest Points to the Origin

6.1 1. Understand the Problem

Input:

A collection of points in the form

(x, y)

and an integer k .

Output:

The k points that are closest to the origin $(0, 0)$.

Requirements:

- Determine the distance of each point from the origin.
- Return exactly k points.
- Compare a sorting-based solution with a heap-based solution.

To compare distances, the algorithm does not need to calculate the actual Euclidean distance. Since

$$distance = \sqrt{x^2 + y^2},$$

and the square root function preserves the ordering of nonnegative values, the squared distance

$$x^2 + y^2$$

can be used instead.

Therefore, a point with a smaller value of $x^2 + y^2$ is closer to the origin.

6.2 2. Design the Solution

Two approaches are used to solve this problem: a sorting-based solution and a heap-based solution.

6.2.1 Sorting-Based Solution

The sorting-based solution first calculates the squared distance of every point from the origin.

Each point is stored together with its distance. The collection is then sorted according to distance in ascending order.

After sorting, the first k points are the k closest points to the origin, so they are added to the result.

For example, given the points

$$(1, 1), \quad (3, 4), \quad (0, 2),$$

their squared distances are

$$2, \quad 25, \quad 4,$$

respectively. Therefore, the points are ordered as

$$(1, 1), \quad (0, 2), \quad (3, 4).$$

The first k points can then be returned.

6.2.2 Heap-Based Solution

The heap-based solution uses a min-heap to keep the points ordered by their distance from the origin.

For each point, its squared distance is calculated and the point is inserted into the min-heap.

The point at the top of the min-heap is always the point with the smallest distance. Therefore, the algorithm repeatedly removes the top point and adds it to the result until k points have been selected.

This approach avoids sorting the entire collection explicitly. However, in this implementation all n points are still inserted into the heap.

6.3 3. Pseudocode

6.3.1 Sorting-Based Solution

Algorithm 7 K Closest Points Using Sorting

```
1: function KCLOSESTUSINGSORTING(points, k)
2:   pointsWithDistances  $\leftarrow$  empty list
3:   for each point in points do
4:     distance  $\leftarrow$  point. $x^2$  + point. $y^2$ 
5:     Add (distance, point) to pointsWithDistances
6:   end for
7:   Sort pointsWithDistances by distance
8:   result  $\leftarrow$  empty list
9:   for i  $\leftarrow$  0 to k - 1 do
10:    Add pointsWithDistances[i].point to result
11:
12:   return result
13: end function
```

6.3.2 Heap-Based Solution

Algorithm 8 K Closest Points Using a Min-Heap

```
1: function KCLOSESTUSINGHEAP(points, k)
2:   minHeap  $\leftarrow$  empty min-heap
3:   for each point in points do
4:     distance  $\leftarrow$  point. $x^2$  + point. $y^2$ 
5:     Insert (distance, point) into minHeap
6:   end for
7:   result  $\leftarrow$  empty list
8:   for i  $\leftarrow$  1 to k do
9:     Add the point at the top of minHeap to result
10:    Remove the top element from minHeap
11:   end for
12:   return result
13: end function
```

6.4 4. Correctness

6.4.1 Sorting-Based Solution

For every point, the algorithm calculates its squared distance from the origin using

$$x^2 + y^2.$$

Because the square root function is increasing for nonnegative values, ordering points by squared distance is equivalent to ordering them by their actual distance from the origin.

The algorithm then sorts all points by their squared distance in ascending order. Therefore, the first point has the smallest distance, the second point has the second-smallest distance, and so on.

Consequently, the first k points after sorting are exactly the k points closest to the origin.

6.4.2 Heap-Based Solution

The min-heap stores every point together with its squared distance from the origin.

The min-heap property guarantees that the point with the smallest distance is always at the top of the heap.

Therefore, when the algorithm removes the top element, it selects the currently closest remaining point. Repeating this process k times selects the k closest points.

Thus, the algorithm correctly returns the required k points.

6.5 5. Complexity Analysis

Let n be the number of points and let k be the number of points requested.

6.5.1 Sorting-Based Solution

Time Complexity:

Calculating the distance for each of the n points takes

$$O(n).$$

Sorting all n points takes

$$O(n \log n).$$

Finally, selecting the first k points takes

$$O(k).$$

Therefore, the total time complexity is

$$O(n) + O(n \log n) + O(k).$$

Since $k \leq n$, the sorting operation dominates, giving

$$O(n \log n).$$

Space Complexity:

The algorithm creates a collection containing all n points together with their distances.

Therefore, the additional space required is

$$O(n).$$

6.5.2 Heap-Based Solution

Time Complexity:

The algorithm calculates the distance of every point and inserts all n points into the min-heap.

Each heap insertion takes

$$O(\log n)$$

time, so inserting all n points takes

$$O(n \log n).$$

The algorithm then removes k points. Each removal takes

$$O(\log n),$$

giving

$$O(k \log n).$$

Therefore, the total time complexity is

$$O(n \log n) + O(k \log n).$$

Since $k \leq n$, this simplifies to

$$O(n \log n).$$

Space Complexity:

The min-heap contains all n points, so the additional space required is

$$O(n).$$

6.5.3 Comparison

For the implementations used in this assignment, both approaches have the same asymptotic time and space complexity:

<i>Approach</i>	<i>Time</i>	<i>ExtraSpace</i>
<i>Sorting</i>	$O(n \log n)$	$O(n)$
<i>Min – Heap</i>	$O(n \log n)$	$O(n)$

The sorting approach explicitly orders all points, while the heap approach maintains the smallest-distance point at the top of the heap.

The heap approach can become more advantageous when only a small number of points k are needed and the heap is limited to size k . However, the implementation used here stores all n points in the heap, so its complexity remains $O(n \log n)$.

In other words, An alternative heap-based approach is to maintain a fixed-size max-heap containing only the k closest points seen so far, which reduces the extra space to $O(k)$ and the time complexity to $O(n \log k)$.

6.6 6. C++ Implementation

6.6.1 Sorting-Based Implementation

The sorting-based implementation stores each point together with its squared distance from the origin. The collection is then sorted, with the distance as the primary sorting value.

After sorting, the first k points are added to the result.

6.6.2 Heap-Based Implementation

The heap-based implementation uses a min-heap. Each heap element contains the squared distance and the corresponding point.

The point at the top of the heap is always the closest remaining point. Therefore, the algorithm removes the top element k times.

6.7 7. Testing

Both implementations were tested using points with different distances from the origin, points located on different axes, and cases where k is small relative to the number of points.

6.7.1 Test Case 1: Basic Example

Input:

$$points = [[1, 3], [-2, 2]]$$

$$k = 1$$

The squared distances are

$$1^2 + 3^2 = 10$$

and

$$(-2)^2 + 2^2 = 8.$$

Expected Output:

$$[[-2, 2]]$$

The point $[-2, 2]$ is closer to the origin.

6.7.2 Test Case 2: Multiple Points

Input:

$$points = [[3, 3], [5, -1], [-2, 4], [0, 1]]$$

$$k = 2$$

The squared distances are

$$18, \quad 26, \quad 20, \quad 1.$$

Therefore, the two closest points are

$$[0, 1], \quad [3, 3].$$

Expected Output:

$$[[0, 1], [3, 3]]$$

6.7.3 Test Case 3: Points on Different Axes

Input:

$$points = [[1, 0], [0, 2], [-1, 1], [3, 0]]$$

$$k = 2$$

The squared distances are

$$1, \quad 4, \quad 2, \quad 9.$$

Therefore, the two closest points are

$$[1, 0], \quad [-1, 1].$$

Expected Output:

$$[[1, 0], [-1, 1]]$$

6.7.4 Edge Case: $k = n$

Input:

$points = [[1, 2], [3, 4], [0, 1]]$

$k = 3$

Since k is equal to the number of points, every point must be returned.

Expected Output:

All three input points.

7 Problem 7: Top K Frequent Elements

7.1 1. Understand the Problem

Input:

An integer array and an integer k .

Output:

The k most frequent values in the array.

Requirements:

- Count how many times each distinct value appears.
- Return the k values with the highest frequencies.
- If two values have the same frequency, the smaller value must appear first.

Tie-Breaking Rule:

When two values have the same frequency, the smaller value is placed first.

For example, if 3 and 5 both appear four times, then 3 should appear before 5 in the result.

7.2 2. Design the Solution

The solution first uses a hash map to count the frequency of every distinct value in the input array.

After counting the frequencies, the algorithm creates a list of pairs. Each pair contains the frequency of a value and the value itself:

$(frequency, value)$.

The list is then sorted using a custom comparison rule.

The primary sorting rule is frequency in descending order, so values with higher frequencies appear first.

If two values have the same frequency, the secondary sorting rule compares their values and places the smaller value first.

After sorting, the first k elements of the list are the k most frequent values.

7.3 3. Pseudocode

Algorithm 9 Find the K Most Frequent Values

```
1: function TOPKFREQUENT(numbers, k)
2:   count  $\leftarrow$  empty hash map
3:   for each number in numbers do
4:     count[number]  $\leftarrow$  count[number] + 1
5:   end for
6:   frequencyList  $\leftarrow$  empty list
7:   for each (number, frequency) in count do
8:     Add (frequency, number) to frequencyList
9:   end for
10:  Sort frequencyList by:
11:    higher frequency first
12:    smaller value first when frequencies tie
13:  result  $\leftarrow$  empty list
14:  for i  $\leftarrow$  0 to k - 1 do
15:    Add frequencyList[i].number to result
16:  end for
17:  return result
18: end function
```

7.4 4. Correctness

The algorithm first counts every occurrence of each distinct value. Therefore, the frequency stored for each value is equal to its actual number of occurrences in the input array.

The algorithm then creates one pair for each distinct value. Each pair contains the value's frequency and the value itself.

The custom comparison rule sorts these pairs so that values with higher frequencies appear first. Therefore, the first k elements after sorting are the k most frequent values.

When two values have the same frequency, the comparison uses their actual values and places the smaller value first. This satisfies the required tie-breaking rule.

Since one pair is created for each distinct value and each pair is selected at most once, the algorithm does not duplicate any values in the result.

Therefore, the first k elements of the sorted list are exactly the required k most frequent values in the required order.

7.5 5. Complexity Analysis

Let n be the number of elements in the input array and let m be the number of distinct values.

Counting:

The algorithm processes every element once to build the frequency map. Therefore, counting takes

$$O(n).$$

Sorting:

There are m distinct values, so the frequency list contains m pairs. Sorting these pairs takes

$$O(m \log m).$$

Selecting the Result:

The algorithm takes the first k elements from the sorted list, which takes

$$O(k).$$

Therefore, the total time complexity is

$$O(n) + O(m \log m) + O(k).$$

Since $k \leq m \leq n$, the overall time complexity is

$$O(n + m \log m).$$

Space Complexity:

The hash map stores one entry for each distinct value, requiring

$$O(m)$$

space.

The frequency list also contains m entries and therefore requires

$$O(m)$$

additional space.

The result contains k values, which requires $O(k)$ space.

Since $k \leq m$, the overall auxiliary space complexity is

$$O(m).$$

7.6 6. C++ Implementation

The implementation first builds a frequency map using an `unordered_map`. It then creates a vector containing pairs of frequency and value.

A custom comparison function is used to sort the pairs. The comparison first checks the frequency. If the frequencies are different, the value with the higher frequency comes first. If the frequencies are equal, the smaller number comes first.

After sorting, the first k values are added to the result.

7.7 7. Testing

The solution was tested using values with different frequencies, values with tied frequencies, negative values, and cases where one value occurs much more frequently than the others.

7.7.1 Test Case 1: Different Frequencies

Input:

$$numbers = [1, 1, 1, 2, 2, 3]$$

$$k = 2$$

The frequencies are:

$$1 \rightarrow 3, \quad 2 \rightarrow 2, \quad 3 \rightarrow 1.$$

Expected Output:

$$[1, 2]$$

7.7.2 Test Case 2: Frequency Tie

Input:

$$numbers = [4, 4, 2, 2, 7, 7, 1]$$

$$k = 3$$

The frequencies are:

$$2 \rightarrow 2, \quad 4 \rightarrow 2, \quad 7 \rightarrow 2, \quad 1 \rightarrow 1.$$

The values 2, 4, and 7 have the same frequency, so the smaller values must appear first.

Expected Output:

$$[2, 4, 7]$$

7.7.3 Test Case 3: Negative Values

Input:

$$numbers = [-1, -1, 2, 2, 2, 3, 3]$$

$$k = 2$$

The frequencies are:

$$2 \rightarrow 3, \quad -1 \rightarrow 2, \quad 3 \rightarrow 2.$$

Since -1 is smaller than 3 , it comes first when the frequencies are tied.

Expected Output:

$$[2, -1]$$

7.7.4 Edge Case: One Distinct Value

Input:

$$numbers = [5, 5, 5, 5]$$

$$k = 1$$

Expected Output:

$$[5]$$

There is only one distinct value, so it is the only possible result.

8 Problem 8: Merge K Sorted Arrays

8.1 1. Understand the Problem

Input:

A collection of k sorted arrays.

Output:

One sorted array containing all elements from the input arrays.

Requirements:

- Each input array is already sorted.
- All elements from all input arrays must appear in the result.
- The final array must be sorted in ascending order.
- The solution should not repeatedly sort the complete output.

8.2 2. Design the Solution

The solution is based on the two-pointer technique used when merging two sorted arrays.

For two arrays, we maintain one pointer for each array. We compare the elements at the two pointers, add the smaller element to the result, and move the pointer of the array that provided that element.

For k arrays, we generalize this idea by maintaining one pointer for each array. Initially, every pointer is at index 0.

At each step, we look at the current element of every array that still has elements remaining. We find the smallest of these current elements and add it to the result. We then move the pointer of the array that provided the smallest element.

We continue this process until all arrays have been completely traversed.

This works because each array is already sorted. Therefore, the smallest remaining element from any array must be at the position currently pointed to in that array.

8.3 3. Pseudocode

Algorithm 10 Merge K Sorted Arrays

```
1: function MERGEKARRAYS(arrays)
2:    $k \leftarrow$  number of arrays
3:   pointers  $\leftarrow$  array of  $k$  zeros
4:   result  $\leftarrow$  empty list
5:   while there are elements remaining in at least one array do
6:     minValue  $\leftarrow$  infinity
7:     minArray  $\leftarrow$  -1
8:     for  $i \leftarrow 0$  to  $k - 1$  do
9:       if pointers[ $i$ ] < length(arrays[ $i$ ]) then
10:        if arrays[ $i$ ][pointers[ $i$ ]] < minValue then
11:          minValue  $\leftarrow$  arrays[ $i$ ][pointers[ $i$ ]]
12:          minArray  $\leftarrow$   $i$ 
13:        end if
14:      end if
15:    end for
16:    Add minValue to result
17:    pointers[minArray]  $\leftarrow$  pointers[minArray] + 1
18:  end while
19:  return result
20: end function
```

8.4 4. Correctness

The algorithm maintains one pointer for each input array. Because every input array is sorted, the pointer always identifies the smallest element that has not yet been added to the result from that array.

At each step, the algorithm compares the current elements of all arrays and selects the smallest one.

Suppose the selected element comes from array i . Every element before the pointer in array i has already been added to the result, and every element after the pointer is greater than or equal to the selected element because the array is sorted.

Similarly, the current elements of all other arrays are greater than or equal to the selected minimum. Therefore, the selected element is the smallest element among all remaining elements.

The algorithm adds this element to the result and advances only the pointer of the array from which the element was selected.

This process continues until every pointer reaches the end of its corresponding array. Therefore, every input element is added exactly once, and each element is added in nondecreasing order.

Thus, the final result contains all elements from all input arrays in sorted order.

8.5 5. Complexity Analysis

Let k be the number of arrays and let n be the total number of elements across all arrays.

Time Complexity:

Every element is added to the result exactly once. Therefore, there are n iterations of the main process.

During each iteration, the algorithm checks the current element of all k arrays to find the smallest one.

This takes

$$O(k)$$

time per element.

Therefore, the total time complexity is

$$O(nk).$$

Space Complexity:

The algorithm uses one pointer for each of the k arrays:

$$O(k).$$

The result itself contains all n elements, requiring

$$O(n)$$

space.

Therefore, including the output, the total space complexity is

$$O(n + k).$$

The auxiliary space used by the algorithm, excluding the output, is

$$O(k).$$

8.6 6. C++ Implementation

The implementation creates one pointer for every input array. Each pointer starts at index 0.

At every step, the implementation checks the current element of every array that still has elements remaining. It keeps track of the smallest current element and the array from which that element came.

The smallest element is added to the result, and only the pointer of that array is increased.

8.7 7. Testing

The solution was tested using multiple sorted arrays, arrays of different sizes, empty arrays, and cases containing duplicate values.

8.7.1 Test Case 1: Three Sorted Arrays

Input:

$$[[1, 4, 7], [2, 5, 8], [3, 6, 9]]$$

Expected Output:

$$[1, 2, 3, 4, 5, 6, 7, 8, 9]$$

At each step, the algorithm selects the smallest current element among the three arrays.

8.7.2 Test Case 2: Arrays of Different Sizes

Input:

$$[[1, 5], [2, 3, 8, 10], [4, 6, 7]]$$

Expected Output:

$$[1, 2, 3, 4, 5, 6, 7, 8, 10]$$

Once an array is exhausted, its pointer is no longer considered. The remaining arrays continue to be processed.

8.7.3 Test Case 3: Duplicate Values

Input:

$[[1, 3, 5], [1, 3, 4], [2, 3, 6]]$

Expected Output:

$[1, 1, 2, 3, 3, 3, 4, 5, 6]$

Duplicate values are preserved because every element from every input array is added to the result exactly once.

8.7.4 Test Case 4: Empty Array

Input:

$[], [1, 2, 3], [4, 5]$

Expected Output:

$[1, 2, 3, 4, 5]$

The empty array is simply skipped because its pointer is already at the end of the array.

8.7.5 Edge Case: All Arrays Empty

Input:

$[], [], []$

Expected Output:

$[]$

There are no elements to merge, so the result is empty.

9 Problem 9: Kth Largest Element

9.1 1. Understand the Problem

Input:

An unsorted integer array and an integer k .

Output:

The k th-largest element in the array.

Requirements:

- The input array is not necessarily sorted.

- Duplicate values count as separate positions.
- Return the value at the k th-largest position.

For example, consider

$$[3, 2, 1, 5, 6, 4]$$

with

$$k = 2.$$

After ordering the values from largest to smallest,

$$[6, 5, 4, 3, 2, 1],$$

the second-largest element is 5.

9.2 2. Design the Solution

Instead of sorting the entire array, the solution only keeps track of the k largest elements seen so far.

A min-heap is useful for this because the smallest element among the values stored in the heap is always at the top.

The algorithm maintains a min-heap with at most k elements.

For every value in the input array:

1. Add the value to the min-heap.
2. If the heap contains more than k elements, remove the smallest element.

After processing all elements, the heap contains the k largest values in the array.

The smallest value among these k elements is the k th-largest element overall. Since the min-heap keeps the smallest of its elements at the top, the top of the heap is the required answer.

9.3 3. Pseudocode

Algorithm 11 Find the Kth Largest Element

```

1: function FINDKTHLARGEST(numbers, k)
2:   minHeap  $\leftarrow$  empty min-heap
3:   for each number in numbers do
4:     Insert number into minHeap
5:     if size of minHeap  $> k$  then
6:       Remove the smallest element from minHeap
7:     end if
8:   end for
9:   return the smallest element in minHeap
10: end function

```

9.4 4. Correctness

The algorithm maintains a min-heap containing at most k elements.

After processing each element, if the heap contains more than k elements, the smallest element is removed. Therefore, the heap always keeps the k largest elements encountered so far.

After all elements have been processed, the heap contains exactly the k largest elements in the entire array, assuming k is a valid position.

Because the heap is a min-heap, its top element is the smallest among these k largest elements.

The smallest value among the k largest values is exactly the k th-largest element in the original array.

Therefore, returning the top element of the min-heap correctly returns the k th-largest element.

9.5 5. Complexity Analysis

Let n be the number of elements in the input array.

Time Complexity:

Each of the n elements is inserted into the min-heap.

The heap contains at most k elements, so each insertion takes

$$O(\log k).$$

When the heap grows larger than k , one element is removed. This also takes

$$O(\log k).$$

Therefore, processing all n elements takes

$$O(n \log k).$$

Space Complexity:

The heap contains at most k elements at any time.

Therefore, the additional space complexity is

$$O(k).$$

9.6 6. C++ Implementation

The implementation uses a min-heap to keep the k largest values seen so far.

For every value in the input array, the value is added to the heap. If the heap becomes larger than k , its smallest value is removed.

At the end, the smallest value among the k largest values is at the top of the heap. This value is the k th-largest element.

9.7 7. Testing

The solution was tested using arrays with different values, duplicate values, and different values of k .

9.7.1 Test Case 1: Basic Example

Input:

$$numbers = [3, 2, 1, 5, 6, 4]$$

$$k = 2$$

The values in descending order are

$$[6, 5, 4, 3, 2, 1].$$

Therefore, the second-largest value is 5.

Expected Output:

$$5$$

9.7.2 Test Case 2: Duplicate Values

Input:

$$numbers = [3, 2, 3, 1, 2, 4, 5, 5, 6]$$

$$k = 4$$

The values in descending order, including duplicates, are

$$[6, 5, 5, 4, 3, 3, 2, 2, 1].$$

Therefore, the fourth-largest value is 4.

Expected Output:

$$4$$

9.7.3 Test Case 3: $k = 1$

Input:

$$numbers = [7, 2, 9, 4, 1]$$

$$k = 1$$

When $k = 1$, the algorithm only needs to keep the largest value.

Expected Output:

$$9$$

9.7.4 Test Case 4: $k = n$

Input:

$$numbers = [4, 1, 7, 3]$$

$$k = 4$$

The fourth-largest value is the smallest value in the array.

Expected Output:

$$1$$

9.7.5 Edge Case: All Values Are Equal

Input:

$$numbers = [5, 5, 5, 5, 5]$$

$$k = 3$$

All positions contain the same value.

Expected Output:

$$5$$

10 Problem 10: Count Inversions

10.1 1. Understand the Problem

Input:

An integer array.

Output:

The total number of inversion pairs in the array.

An inversion is a pair of indices (i, j) such that

$$i < j$$

and

$$values[i] > values[j].$$

Duplicate values do not form inversions because the condition requires a strictly greater value.

For example, in

$$[2, 4, 1, 3, 5],$$

the inversions are

$$(2, 1), \quad (4, 1), \quad (4, 3).$$

Therefore, the array contains 3 inversions.

10.2 2. Design the Solution

A straightforward solution would compare every pair of elements and check whether it forms an inversion. However, this would require checking $O(n^2)$ pairs.

Instead, the solution modifies Merge Sort to count inversions efficiently.

The main idea comes from the way Merge Sort divides the array into two halves. For every possible pair of values, there are three possibilities:

1. Both values are in the left half.
2. Both values are in the right half.
3. One value is in the left half and the other is in the right half.

The recursive calls count the inversions that occur entirely within the left and right halves.

The merge step then counts the inversions that cross from the left half into the right half.

During the merge, both halves are already sorted. If the current value in the left half is greater than the current value in the right half, then the current right value is smaller than every remaining value in the left half.

Therefore, instead of counting these inversions one at a time, we can count all of them at once.

10.3 3. Pseudocode

10.3.1 Merge and Count

Algorithm 12 Merge Two Sorted Halves and Count Inversions

```
1: function MERGEANDCOUNT(values, temporary, left, middle, right)
2:    $i \leftarrow left$ 
3:    $j \leftarrow middle + 1$ 
4:    $position \leftarrow left$ 
5:    $inversions \leftarrow 0$ 
6:   while  $i \leq middle$  and  $j \leq right$  do
7:     if  $values[i] \leq values[j]$  then
8:        $temporary[position] \leftarrow values[i]$ 
9:        $i \leftarrow i + 1$ 
10:    else
11:       $temporary[position] \leftarrow values[j]$ 
12:       $j \leftarrow j + 1$ 
13:       $inversions \leftarrow inversions + (middle - i + 1)$ 
14:    end if
15:     $position \leftarrow position + 1$ 
16:  end while
17:  while  $i \leq middle$  do
18:     $temporary[position] \leftarrow values[i]$ 
19:     $i \leftarrow i + 1$ 
20:     $position \leftarrow position + 1$ 
21:  end while
22:  while  $j \leq right$  do
23:     $temporary[position] \leftarrow values[j]$ 
24:     $j \leftarrow j + 1$ 
25:     $position \leftarrow position + 1$ 
26:  end while
27:  Copy  $temporary[left \dots right]$  back into  $values$ 
28:  return  $inversions$ 
29: end function
```

10.3.2 Merge Sort and Count

Algorithm 13 Merge Sort and Count Inversions

```

1: function MERGESORTANDCOUNT(values, temporary, left, right)
2:   if left ≥ right then
3:     return 0
4:   end if
5:   middle ← left + (right − left)/2
6:   inversions ← 0
7:   inversions ← inversions + MERGESORTAND-
COUNT(values, temporary, left, middle)
8:   inversions ← inversions + MERGESORTAND-
COUNT(values, temporary, middle + 1, right)
9:   inversions ← inversions + MERGEAND-
COUNT(values, temporary, left, middle, right)
10:  return inversions
11: end function

```

10.4 4. Correctness

The algorithm counts all inversions by dividing them into three categories.

First, the recursive call on the left half counts every inversion where both indices belong to the left half.

Second, the recursive call on the right half counts every inversion where both indices belong to the right half.

Finally, the merge step counts every inversion where the first value belongs to the left half and the second value belongs to the right half.

Because the two halves are sorted before they are merged, whenever the current left value is greater than the current right value, that right value is smaller than every remaining value in the left half.

Therefore, all remaining left-half values form inversions with the current right-half value, and the algorithm correctly adds

$$middle - i + 1$$

to the inversion count.

These three categories contain every possible pair of indices exactly once. Therefore, the total returned by the algorithm is exactly the number of inversions in the original array.

10.5 5. Complexity Analysis

Let n be the number of elements in the array.

At each level of Merge Sort, the algorithm processes all n elements while performing the merge operations. Therefore, each level takes

$$O(n)$$

time.

Merge Sort has

$$O(\log n)$$

levels.

Therefore, the overall time complexity is

$$O(n \log n).$$

This is an improvement over the straightforward pairwise comparison approach, which requires

$$O(n^2)$$

time.

Space Complexity:

The algorithm creates a temporary array of size n to perform the merge operations.

Therefore, the auxiliary space complexity is

$$O(n).$$

The recursion also uses $O(\log n)$ stack space, which is dominated by the $O(n)$ temporary array.

10.6 6. C++ Implementation

The implementation uses two functions.

The `mergeAndCount` function merges two sorted halves and counts inversions that cross from the left half to the right half.

The `mergeSortAndCount` function recursively divides the array, counts inversions in each half, and then uses `mergeAndCount` to count the inversions between the two halves.

10.7 7. Testing

The solution was tested using arrays that are already sorted, arrays sorted in reverse order, arrays containing duplicate values, and arrays containing a mixture of ordered and inverted pairs.

10.7.1 Test Case 1: Basic Example

Input:

[2, 4, 1, 3, 5]

The inversions are

(2, 1), (4, 1), (4, 3).

Therefore,

Expected Output: 3

10.7.2 Test Case 2: Already Sorted

Input:

[1, 2, 3, 4, 5]

There are no pairs where an earlier value is greater than a later value.

Expected Output: 0

10.7.3 Test Case 3: Reverse Sorted

Input:

[5, 4, 3, 2, 1]

Every pair of positions forms an inversion.

The number of inversions is

$$\frac{5(5-1)}{2} = 10.$$

Expected Output: 10

10.7.4 Test Case 4: Duplicate Values

Input:

[1, 3, 2, 3, 1]

The inversions are

(3, 2), (3, 1), (3, 1), (2, 1).

Therefore,

Expected Output: 4

Equal values are not counted as inversions because the condition requires

$$values[i] > values[j].$$

10.7.5 Edge Case: Empty Array

Input:

`[]`

There are no pairs of elements, so there are no inversions.

Expected Output: 0